2

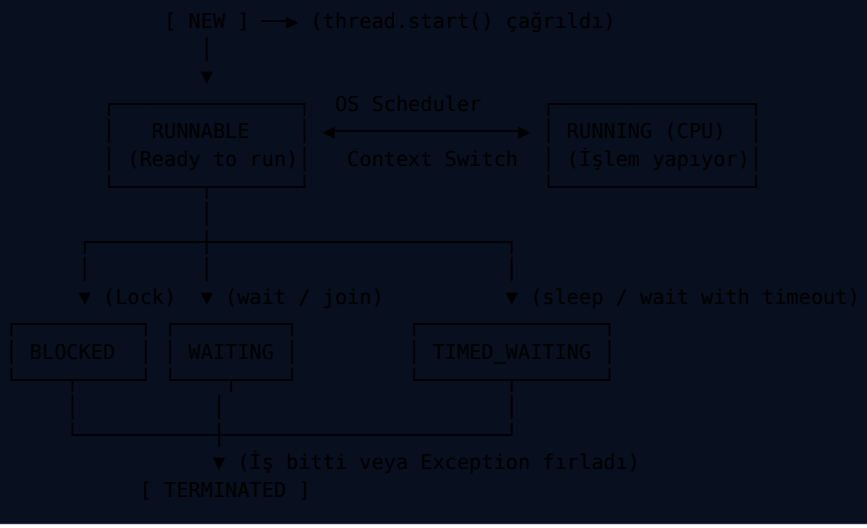
BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Java Thread Yaşam Döngüsü: 6 Kritik Durum (State Transitions)

Java Thread Lifecycle: The 6 Core JVM States & State Transitions

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir işçinin gün içindeki 6 hali vardır: 1) NEW: İşe yeni alındı, masasında oturuyor ama henüz 'Başla' denmedi. 2) RUNNABLE: Masasında hazır bekliyor veya harıl harıl çalışıyor. 3) BLOCKED: Tuvalet kapısının önünde bekliyor çünkü içeride biri var (Kilit kavgası). 4) WAITING: Patron 'Ben sana haber verene kadar kenarda bekle' dedi (Süresiz bekleme). 5) TIMED_WAITING: '20 dakika öğle uykusuna yat' dendi (Süreli bekleme). 6) TERMINATED: Mesaisi bitti, evine gitti!

🇬🇧 ENGLISH (ELI5)

An employee's lifecycle stages: 1) NEW: Hired and given a badge, but hasn't clocked in yet. 2) RUNNABLE: Either actively typing at the desk or standing ready for an assignment. 3) BLOCKED: Waiting outside the restroom because a colleague locked the door from inside (Lock Contention). 4) WAITING: Sitting in the lounge waiting indefinitely for a manager's signal. 5) TIMED_WAITING: Taking an exact 15-minute coffee break. 6) TERMINATED: Shift finished, clocked out forever!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Thread.State enum değerleri ve mülakat derinliği: 1) NEW: `new Thread()` yapılmış ancak `start()` çağrılmamıştır. `start()` yerine doğrudan `run()` çağrılırsa yeni thread açılmaz, çağıran thread içinde senkron çalışır! 2) RUNNABLE: JVM seviyesinde thread koşmaya hazırdır veya CPU'da çalışıyordur. JVM, işletim sisteminin 'Ready' ve 'Running' durumlarını tek bir RUNNABLE altında toplar. 3) BLOCKED: Thread bir `synchronized` bloğuna girmek için Monitor Lock (Intrinsic Lock) beklemektedir. 4) WAITING: `Object.wait()`, `Thread.join()` veya `LockSupport.park()` ile süresiz askıya alınmıştır; başka bir thread `notify()` veya `unpark()` çağırana kadar CPU tüketmez. 5) TIMED_WAITING: `Thread.sleep(ms)`, `wait(ms)` veya `parkNanos()` ile zaman sınırıyla bekler. 6) TERMINATED: `run()` metodu doğal yolla bitmiş veya yakalanmamış bir exception ile sonlanmıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Thread State nuances: 1) NEW to RUNNABLE: Calling `start()` triggers OS `pthread_create()`. Invoking `run()` directly bypasses thread creation, executing synchronously on the caller's stack. 2) RUNNABLE Collapsing: The JVM intentionally does not distinguish between OS 'Ready' (in kernel runqueue) and OS 'Executing' (on CPU core); both report as RUNNABLE. 3) BLOCKED vs WAITING: A critical distinction in thread dumps! BLOCKED strictly means waiting for a synchronized monitor lock. WAITING means parked via `LockSupport.park()` or `Object.wait()`, awaiting a signal. 4) TERMINATED: Dead threads cannot be restarted. Calling `start()` on a TERMINATED thread throws `IllegalThreadStateException`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Thread Durum Geçişlerini İnceleme
void main() throws InterruptedException {
    final Object lock = new Object();

    Thread worker = new Thread(() -> {
        try {
            Thread.sleep(100); // TIMED_WAITING
            synchronized (lock) {
                lock.wait();   // WAITING
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    });

    IO.println("1. Durum: " + worker.getState()); // NEW
    worker.start();
    IO.println("2. Durum: " + worker.getState()); // RUNNABLE

    Thread.sleep(50);
    IO.println("3. Durum (Sleep anı): " + worker.getState()); // TIMED_WAITING

    Thread.sleep(100);
    synchronized (lock) {
        IO.println("4. Durum (Wait anı): " + worker.getState()); // WAITING
        lock.notify();
    }

    worker.join();
    IO.println("5. Durum (Bitti): " + worker.getState()); // TERMINATED
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'ReentrantLock bekleyen bir thread hangi duruma geçer?' Yanıt: BLOCKED DEĞİL, WAITING (veya TIMED_WAITING) durumuna geçer! Çünkü ReentrantLock `LockSupport.park()` kullanır; BLOCKED durumu SADECE `synchronized` monitor lock için geçerlidir!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Classic Interview Trap: 'What state is a thread in when blocked on a ReentrantLock?' Answer: 'WAITING (or TIMED_WAITING), NOT BLOCKED! ReentrantLock is built on AbstractQueuedSynchronizer (AQS) using LockSupport.park(). BLOCKED is strictly reserved for intrinsic synchronized monitors.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- JVM'de 6 thread durumu vardır: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
- BLOCKED sadece synchronized içindir; ReentrantLock bekleyen thread WAITING olur.
- Ölmüş (TERMINATED) bir thread yeniden `start()` edilemez.

🎯 Key Takeaways for the Interview

- JVM threads traverse 6 states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
- BLOCKED is exclusive to synchronized; AQS locks park threads in WAITING state.
- A TERMINATED thread cannot be restarted; doing so throws IllegalThreadStateException.